

# Chapter 11 - Hibernate & JPA Annotations

## 1. Introduction

Annotations are metadata used to map Java classes and fields to database tables and columns. Modern Hibernate applications primarily use annotations instead of XML mappings.

| Annotation         | Purpose                                  |
|--------------------|------------------------------------------|
| @Entity            | Marks a class as a JPA entity            |
| @Table             | Maps entity to a table                   |
| @Id                | Marks the primary key                    |
| @GeneratedValue    | Defines primary key generation strategy  |
| @Column            | Maps a field to a table column           |
| @Transient         | Excludes a field from persistence        |
| @Temporal          | Maps java.util.Date/Calendar (legacy)    |
| @Enumerated        | Stores enum values                       |
| @Lob               | Maps large object (CLOB/BLOB)            |
| @Version           | Enables optimistic locking               |
| @CreationTimestamp | Stores creation timestamp (Hibernate)    |
| @UpdateTimestamp   | Stores last update timestamp (Hibernate) |

## 2. Entity Mapping Example

```
@Entity
@Table(name="EMPLOYEE")
public class Employee {

    @Id
    @GeneratedValue
    private Integer id;

    @Column(name="EMP_NAME")
    private String name;

    @Transient
    private String confirmPassword;
}
```

### @Entity

Marks a Java class as a persistent entity. Hibernate creates or maps it to a database table.

### @Table

Specifies the database table name. If omitted, Hibernate typically uses the entity name as the default table name.

### @Id

Identifies the primary key column. Every entity must have an identifier.

## **@GeneratedValue**

Specifies how the primary key value is generated. Common strategies include AUTO, IDENTITY, SEQUENCE and TABLE.

## **@Column**

Customizes column mapping, including name, length, nullable and unique constraints.

## **@Transient**

Excludes a field from database persistence. The field exists only in memory and is not written to or read from the database.

## **@Enumerated**

Maps enum values using EnumType.STRING or EnumType.ORDINAL. STRING is generally preferred because it is easier to maintain.

## **@Lob**

Maps large binary or character data such as images, PDFs or long text.

## **@Version**

Supports optimistic locking by maintaining a version column. Hibernate checks this value during updates to detect concurrent modifications.

## **@CreationTimestamp & @UpdateTimestamp**

Hibernate-specific annotations that automatically populate creation and last-update timestamps.

## **Best Practices**

- Prefer Integer/Long wrapper types for identifiers.
- Use EnumType.STRING instead of ORDINAL in most cases.
- Use @Transient for non-persistent fields.
- Use @Version for concurrent update protection.

## **Interview Questions**

1. Difference between @Entity and @Table? 2. What is @Transient? 3. Explain @GeneratedValue strategies. 4. Why use @Version? 5. STRING vs ORDINAL in @Enumerated?